

Лабораторная работа № 2 SAST (Static Application Security Testing)

Раздел 1. ЦЕЛЬ РАБОТЫ

Освоить методику статического анализа безопасности кода (SAST) на примере FastAPI-приложения с использованием инструментов Bandit, Safety и Semgrep, а также интерпретировать полученные результаты и предложить меры по устранению уязвимостей.

Раздел 2. ПЕРЕЧЕНЬ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1. Python 3.10+ — среда выполнения.
2. Bandit — SAST-инструмент для поиска уязвимостей в Python-коде .
3. Safety — сканер уязвимостей зависимостей Python .
4. Semgrep — многоязычный SAST-инструмент с поддержкой FastAPI .
5. pip-audit (опционально) — аудит зависимостей на известные CVE.
6. Текстовый редактор — для просмотра кода и отчетов.
7. Браузер — для просмотра HTML-отчетов.

Раздел 3. ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

Исходные данные

Разработанное ранее FastAPI-приложение с эндпоинтами /, /health, /cpu, /metrics. В проекте присутствуют файлы main.py, requirements.txt, возможно, другие модули. Приложение содержит преднамеренные или реальные уязвимости, которые требуется обнаружить с помощью SAST-инструментов.

Этап 1. Подготовка окружения и установка инструментов

1. Перейдите в корневую директорию проекта:

```
cd fastapi-app
```

2. Установите инструменты SAST:

```
pip install bandit safety semgrep
```

3. Проверьте установку:

```
bandit --version
```

```
safety --version
```

```
semgrep --version
```

4. Создайте структуру для отчетов:

```
mkdir -p reports
```

Этап 2. Запуск Bandit — статический анализ кода FastAPI

Bandit сканирует Python-код на наличие известных уязвимостей: hardcoded credentials, SQL-инъекции, вызовы опасных функций (eval, exec) и др. .

1. Запустите Bandit с рекурсивным обходом всех файлов:

```
bandit -r ./app -f txt
```

*Где ./app — путь к директории с исходным кодом (может отличаться)

2. Сохраните отчет в JSON-формате:

```
bandit -r ./app -f json -o reports/bandit-report.json
```

3. Запустите анализ только с уровнями HIGH и MEDIUM (без LOW):

```
bandit -r ./app -f txt --severity-level medium
```

4. Сгенерируйте HTML-отчет для просмотра в браузере:

```
bandit -r ./app -f html -o reports/bandit-report.html
```

5. Проанализируйте отчет:

- Обратите внимание на High и Medium уязвимости.

- Типичные находки для FastAPI-приложений :

- B501: SSL-проверка отключена (например, в HTTP-запросах к внешним API).

- B108: Использование небезопасной временной директории /tmp.

- B101: assert в production-коде.

Этап 3. Запуск Safety — сканирование зависимостей

Safety проверяет файлы зависимостей (requirements.txt, Pipfile, pyproject.toml) на наличие известных уязвимостей в пакетах .

1. Запустите сканирование всех зависимостей:

safety scan

Safety просканирует все файлы зависимостей в директории проекта.

2. Если требуется аутентификация, выполните вход:

safety auth

3. Запустите сканирование с экспортом в JSON:

safety scan --output json > reports/safety-report.json

4. Проанализируйте результаты:

- Уязвимые версии пакетов (например, FastAPI, Uvicorn, Starlette).
- Рекомендации по обновлению до защищенных версий.

5. (Опционально) Проверка через pip-audit:

pip-audit -r requirements.txt --strict

Этап 4. Запуск Semgrep — расширенный SAST для FastAPI

Semgrep поддерживает анализ с учетом фреймворков (FastAPI, Flask, Django) и может обнаруживать специфические для фреймворка уязвимости .

1. Запустите Semgrep со стандартными правилами:

semgrep scan ./app --config p/default

2. Запустите с правилами безопасности (p/security-audit):

semgrep scan ./app --config p/security-audit -o reports/semgrep-report.json --json

3. Запустите Semgrep с правилами для FastAPI (Pro-движок, если доступен):

semgrep login

semgrep ci

4. Изучите найденные уязвимости:

- Небезопасное использование глобальных объектов.
- Неправильная валидация входных данных.
- Проблемы с потоком данных от пользовательского ввода к чувствительным функциям.

Этап 5. Интерпретация результатов и меры по устранению

1. Составьте сводную таблицу найденных уязвимостей:

Инструмент	Уязвимость	Файл	Строка	Рекомендация
Bandit	B501: SSL verification disabled	scanner.py	31	Включить verify=True или использовать SSLContext
Safety	CVE-2023-XXXXX: FastAPI < 0.100	requirements.txt	—	Обновить до версии 0.100+
Semgrep	Unsafe use of eval on user input	routes.py	44	Заменить на безопасный парсер (json, ast.literal_eval)
Bandit	B101: assert used in production	main.py	27	Удалить assert, использовать exception handling

2. Выполните исправления в коде:

- Замените небезопасные вызовы (eval → ast.literal_eval).
- Добавьте валидацию входных данных через Pydantic.
- Обновите уязвимые зависимости в requirements.txt.
- Добавьте securityContext для контейнеров.

3. Проверьте исправления повторным запуском SAST:

bandit -r ./app -f txt

safety scan

4. Настройте Bandit через конфигурационный файл для исключения ложных срабатываний:

Создайте .bandit в корне проекта:

[bandit]

```
exclude_dirs = ['tests', 'venv']
```

```
skips = ['B101'] # Пропустить assert, если используется для type-narrowing
```

Этап 6. Интеграция SAST в CI/CD

1. Настройка Pre-commit хука для Bandit:

Добавьте в .pre-commit-config.:

```
repos:
```

```
- repo: https://github.com/PyCQA/bandit
```

```
  rev: 1.9.4
```

```
  hooks:
```

```
    - id: bandit
```

```
      args: ["-ll"] # Уровень LOW и выше
```

2. Настройка Safety в GitLab CI (или GitHub Actions):

Пример для GitLab CI

```
security-scan:
```

```
  script:
```

```
    - pip install safety
```

```
    - safety scan
```

3. Настройка Semgrep в CI:

```
semgrep:
```

```
  script:
```

```
    - semgrep ci
```

Раздел 4. ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОГО ВЫПОЛНЕНИЯ

Задание 1. Глубокий анализ Bandit.

1. Запустите Bandit с уровнями HIGH и MEDIUM.

2. Для каждого найденного HIGH-уровня укажите конкретную строку кода и предложите исправление.

3. Создайте конфигурационный файл .bandit, исключив из сканирования папки tests и venv .

Задание 2. Анализ зависимостей.

1. Выполните safety scan для вашего проекта.

2. Найдите минимум 3 уязвимости в зависимостях (если они есть) или предложите потенциально уязвимые пакеты для тестирования.

3. Предложите пути обновления для каждой найденной уязвимости.

Задание 3. Исправление уязвимостей.

1. Создайте файл app/utils.py с функцией, использующей eval для вычислений.

2. Запустите Bandit и найдите проблему.

3. Исправьте код, заменив eval на ast.literal_eval или другой безопасный метод.

4. Проверьте исправление повторным запуском.

Раздел 5. КОНТРОЛЬНЫЕ ВОПРОСЫ ДЛЯ ЗАЩИТЫ ОТЧЕТА

1. SAST: Что такое статический анализ безопасности кода и чем он отличается от динамического (DAST)?

2. Bandit: Какие основные категории уязвимостей обнаруживает Bandit и как его интегрировать в CI/CD?

3. Safety: Какую задачу решает Safety и почему важно сканировать не только код, но и зависимости?

4. Semgrep: В чем преимущество Semgrep перед Bandit при анализе FastAPI-приложений и как работает фреймворк-специфичный анализ?

5. Ложные срабатывания: Как бороться с ложными срабатываниями (False Positives) в SAST-инструментах? Приведите пример.

6. OWASP Top 10: Какие категории из OWASP Top 10 можно обнаружить с помощью SAST?